

Programação Orientada a Objetos (POO)

É um paradigma de programação baseado no conceito de **objetos**, que são instâncias de **classes**. Visa organizar e modelar software de forma a simular comportamentos do mundo real, dividindo o sistema em "objetos" que possuem características e comportamentos. A POO promove a reutilização, modularidade e flexibilidade no desenvolvimento de software.

- **Objetos**: são instâncias de classes, representam entidades do mundo real. Cada objeto possui atributos (também chamados de propriedades ou campos) que armazenam o estado ou as características do objeto e métodos (ou funções) que definem o comportamento desse objeto.

- **Classes**: são modelos ou moldes para a criação de objetos. Definem as propriedades e os comportamentos que os objetos dessa classe terão.

A classe é uma definição do objeto, e o objeto é uma instância da classe.

- **Encapsulamento** : é o conceito de ocultar os detalhes internos de um objeto e permitir que ele interaja com o mundo exterior apenas através de interfaces bem definidas. Isso é alcançado tornando os atributos do objeto privados (ou protegidos) e fornecendo métodos públicos (como getters e setters) para acessar e modificar esses atributos. O encapsulamento ajuda a proteger os dados e a reduzir a complexidade do sistema.

- **Herança**: permite que uma classe herde atributos e métodos de outra classe, criando uma hierarquia entre as classes. A classe que herda é chamada de subclasse (ou classe derivada), enquanto a classe da qual ela herda é chamada de superclasse (ou classe base). A herança promove a reutilização de código, pois a subclasse pode usar os membros da superclasse sem ser necessário reescrevê-los.

- **Polimorfismo**: permite que objetos de diferentes tipos sejam tratados de maneira uniforme, ou que um método tenha diferentes comportamentos dependendo do tipo do objeto. Pode ser estático (também chamado de sobrecarga de método), onde o método tem o mesmo nome, mas recebe diferentes parâmetros, ou dinâmico (também chamado de sobrescrição de método), onde um método na subclasse substitui um método da classe base. O polimorfismo ajuda a aumentar a flexibilidade e extensibilidade do sistema.

- **Abstração**: permite esconder a complexidade e mostrar apenas os aspectos essenciais de um objeto ou sistema. Em C#, a abstração pode ser alcançada através de interfaces e classes abstratas. Uma classe abstrata pode ter métodos com ou sem implementação, enquanto uma interface só define métodos que devem ser implementados pelas classes que a implementam.

Modificadores de Acesso

Controlam a visibilidade e a acessibilidade de tipos (como classes, métodos, propriedades, campos, etc.) em relação a outras partes do código. Garantem a segurança, o encapsulamento e o controle de acesso adequado dentro de um programa.

Modificador	Visibilidade	Acesso
<code>private</code>	Apenas dentro da classe onde o membro é declarado	Não acessível fora da classe
<code>protected</code>	Dentro da classe e suas subclasses	Acessível dentro da classe e nas classes derivadas
<code>internal</code>	Dentro do mesmo assembly	Acessível apenas dentro do mesmo assembly (projeto ou biblioteca)
<code>public</code>	Em qualquer lugar	Acessível de qualquer parte do código (fora da classe, fora do assembly)

Modificador new

Quando se usa o `new`, ele oculta o método da classe base, ao referenciar o objeto com o tipo da classe base, o método da classe base será chamado, e não o da classe derivada.

-**Virtual**: Colocado no método da classe Pai para autorizar a substituição.

- **Override**: Colocado no método da classe Filha para substituir o comportamento do Pai. Mantém o Polimorfismo.

- New (Shadowing): Oculta o membro do pai sem o substituir. Se referenciar a classe pelo tipo do Pai, o C# ignora o seu new e usa o valor original do Pai.

Ex:

```
class Animal {
    public virtual void EmitirSom() => Console.WriteLine("Som genérico");
    public void Info() => Console.WriteLine("Eu sou um Animal");
}

class Cao : Animal {
    // Substitui o comportamento (Polimorfismo Dinâmico)
    public override void EmitirSom() => Console.WriteLine("Ladrar");

    // Oculta o comportamento (Shadowing/Estático)
    public new void Info() => Console.WriteLine("Eu sou um Cão");
}

// NO PROGRAMA:
Animal meuPet = new Cao();

meuPet.EmitirSom(); // Output: Ladrar (Override vence sempre)
meuPet.Info();      // Output: Eu sou um Animal (New obedece ao tipo da variável)
```

Polimorfismo e tipagem específica

Aspecto	Produto produto = new ProdutoEletronico();	ProdutoEletronico produto = new ProdutoEletronico();
Flexibilidade	Maior flexibilidade e possibilidade de adicionar novos tipos de produtos sem modificar o código existente.	Menos flexível, pois o código fica acoplado a um tipo específico (ProdutoEletronico).
Acesso aos membros da subclasse	Acesso limitado aos membros específicos de ProdutoEletronico sem casting.	Acesso direto a todos os membros específicos de ProdutoEletronico.
Polimorfismo	Uso de polimorfismo, permitindo tratar diferentes tipos de produtos de forma uniforme.	Não usa polimorfismo; o código é específico para ProdutoEletronico.
Facilidade de manutenção	Pode ser mais fácil de manter e estender, pois o código é mais genérico.	O código pode ser mais difícil de modificar para outros tipos de Produto.
Segurança	Requer casting para aceder a membros da subclasse, o que pode causar erros se não for bem feito.	Menos necessidade de casting, tornando o código mais seguro e direto.

- **base()**: Quando uma classe deriva de outra, o construtor da classe filha deve, idealmente, chamar o construtor da classe pai usando : base(). Isso garante que as propriedades herdadas sejam inicializadas corretamente pela classe que as definiu originalmente.